

Linux Fundamentals Day 4 — Interview-Oriented

(Sreelekha v u-30 sept 2026)

Machine Test

Task 1 — User Information Investigation

Execute: whoami, id, groups.

```
sreelekha@LAPTOP-CKI0PC9R: ~  
sreelekha@LAPTOP-CKI0PC9R:~$ whoami  
sreelekha  
sreelekha@LAPTOP-CKI0PC9R:~$ id  
uid=1000(sreelekha) gid=1000(sreelekha) groups=1000(sreelekha),4(adm),20(dialout),24(cdrom),25(floppy),27(sudo),29(audio),30(dip),44(video),46(plugdev),100(users),105(netdev)  
sreelekha@LAPTOP-CKI0PC9R:~$ groups  
sreelekha adm dialout cdrom floppy sudo audio dip video plugdev users netdev  
sreelekha@LAPTOP-CKI0PC9R:~$
```

Questions:

1. What is your current username?

The username displayed by the whoami command.(sreelekha)

2. Difference between UID and GID?

- **UID (User ID):** A unique number assigned to a user.
- **GID (Group ID):** A unique number assigned to a group.

3. List all groups your user belongs to.

```
sreelekha@LAPTOP-CKI0PC9R: ~  
sreelekha@LAPTOP-CKI0PC9R:~$ whoami  
sreelekha  
sreelekha@LAPTOP-CKI0PC9R:~$ id  
uid=1000(sreelekha) gid=1000(sreelekha) groups=1000(sreelekha),4(adm),20(dialout),24(cdrom),25(floppy),27(sudo),29(audio),30(dip),44(video),46(plugdev),100(users),105(netdev)  
sreelekha@LAPTOP-CKI0PC9R:~$ groups  
sreelekha adm dialout cdrom floppy sudo audio dip video plugdev users netdev  
sreelekha@LAPTOP-CKI0PC9R:~$
```

4. Which group allows administrative access?

Usually the sudo **group** allows administrative access on Ubuntu. Users in this group can use sudo to execute commands with administrator privileges.

Task 2 — Linux User Database

Execute: `cat /etc/passwd`

```
sreelekha@LAPTOP-CKI0PC9R:~$ cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin
_apt:x:42:65534::/nonexistent:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
systemd-network:x:998:998:systemd Network Management:/:/usr/sbin/nologin
dhcpcd:x:996:996:DHCP Client Daemon:/usr/lib/dhcpcd:/bin/false
messagebus:x:995:995:System Message Bus:/nonexistent:/usr/sbin/nologin
syslog:x:100:101::/nonexistent:/usr/sbin/nologin
systemd-resolve:x:989:989:systemd Resolver:/:/usr/sbin/nologin
_chrony:x:988:988:Chrony Daemon:/var/lib/chrony:/usr/sbin/nologin
landscape:x:101:103::/var/lib/landscape:/usr/sbin/nologin
polkitd:x:987:987:User for polkitd:/:/usr/sbin/nologin
sreelekha:x:1000:1000:/:/home/sreelekha:/bin/bash
```

Questions:

1. Which file stores Linux user information?

`/etc/passwd` stores basic information about Linux users.

2. Explain `ubuntu:x:1000:1000:Ubuntu User:/home/ubuntu:/bin/bash`

Each field is separated by `:`.

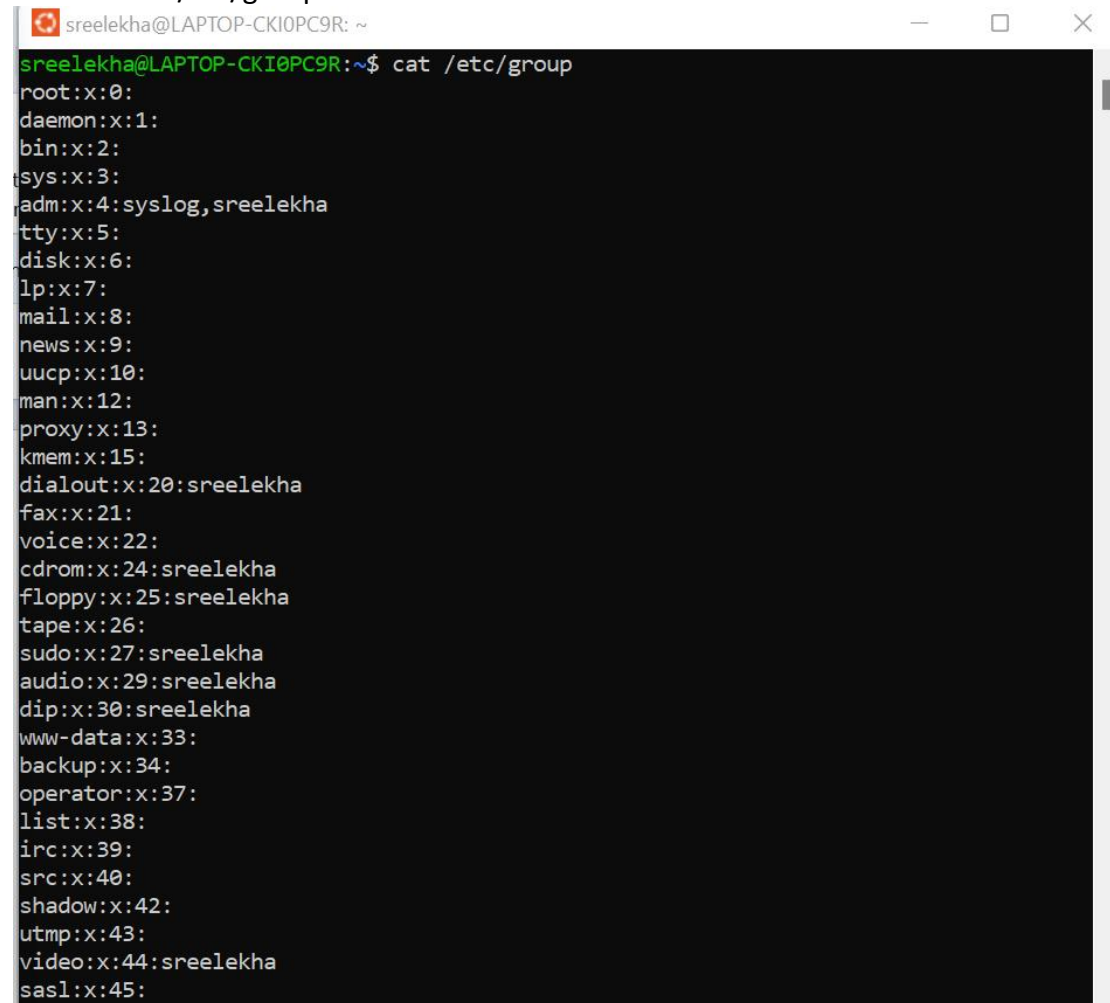
- **ubuntu** → Username
- **x** → Password placeholder; the actual password hash is stored in `/etc/shadow`
- **1000** → UID (User ID)
- **1000** → GID (Group ID)
- **Ubuntu User** → User description/comment
- **/home/ubuntu** → User's home directory
- **/bin/bash** → User's login shell

3. What is the purpose of the login shell field?

The login shell field specifies the **shell/program that is started when the user logs in**. For example, `/bin/bash` means the user gets the Bash shell.

Task 3 — Linux Group Database

Execute: `cat /etc/group`

A terminal window titled 'sreelekha@LAPTOP-CKI0PC9R: ~' with standard window controls. The command 'cat /etc/group' has been executed, displaying the contents of the group database file. The output lists system and user groups with their respective IDs and members.

```
sreelekha@LAPTOP-CKI0PC9R:~$ cat /etc/group
root:x:0:
daemon:x:1:
bin:x:2:
sys:x:3:
adm:x:4:syslog,sreelekha
tty:x:5:
disk:x:6:
lp:x:7:
mail:x:8:
news:x:9:
uucp:x:10:
man:x:12:
proxy:x:13:
kmem:x:15:
dialout:x:20:sreelekha
fax:x:21:
voice:x:22:
cdrom:x:24:sreelekha
floppy:x:25:sreelekha
tape:x:26:
sudo:x:27:sreelekha
audio:x:29:sreelekha
dip:x:30:sreelekha
www-data:x:33:
backup:x:34:
operator:x:37:
list:x:38:
irc:x:39:
src:x:40:
shadow:x:42:
utmp:x:43:
video:x:44:sreelekha
sasl:x:45:
```

Questions:

1. Which file stores Linux group information?

`/etc/group`

2. What is the purpose of a group in Linux?

To organize users and manage permissions and access to files ,directories, and system resources

3. Explain `docker:x:999:ubuntu`

- **docker** → Group name
- **x** → Group password placeholder
- **999** → GID (Group ID)
- **ubuntu** → User belonging to the docker group

So, **ubuntu** is a **member of the docker group**, which can allow the user to interact with Docker according to the permissions configured for that group.

Task 4 — Permission Analysis

Create employee.txt and analyze -rw-r--r--.

```
sreelekha@LAPTOP-CKI0PC9R:~$ touch employee.txt
sreelekha@LAPTOP-CKI0PC9R:~$ ls -l employee.txt
-rw-r--r-- 1 sreelekha sreelekha 0 Oct  1 11:23 employee.txt
sreelekha@LAPTOP-CKI0PC9R:~$
```

Questions:

Owner permissions, Group permissions, Others permissions, and file type meaning.

1. Owner permissions:

rw- → Read and Write permissions.

2. Group permissions:

r-- → Read-only permission.

3. Others permissions:

r-- → Read-only permission.

4. File type meaning:

- → It represents a **regular file**.

Task 5 — Numeric Permission Conversion

Convert `rw-rw-rwx`, `rw-r-xr-x`, `rw-r--r--`, `rw-----`, `r-xr-xr-x` to numeric values.

- **r = 4** → Read
- **w = 2** → Write
- **x = 1** → Execute
- **- = 0** → No permission

1. `rw-rw-rwx`

- Owner: $rwx = 4+2+1 = 7$
- Group: $rwx = 4+2+1 = 7$
- Others: $rwx = 4+2+1 = 7$

Answer: 777

2. `rw-r-xr-x`

- Owner: $rwx = 7$
- Group: $r-x = 4+1 = 5$
- Others: $r-x = 4+1 = 5$

Answer: 755

3. `rw-r--r--`

- Owner: $rw- = 4+2 = 6$
- Group: $r-- = 4$
- Others: $r-- = 4$

Answer: 644

4. `rw-----`

- Owner: $rw- = 6$
- Group: $--- = 0$
- Others: $--- = 0$

Answer: 600

5. `r-xr-xr-x`

- Owner: $r-x = 4+1 = 5$
- Group: $r-x = 5$
- Others: $r-x = 5$ ====> so **Answer: 555**

Explain why 777 is dangerous and why 600 is recommended.

777 gives **read, write, and execute permissions to everyone** — owner, group, and others. This can allow unauthorized users to modify or execute files, creating security risks

600 gives **read and write permission only to the owner**. Group and others have no permissions.

600 = rw-----

This is commonly suitable for **private files containing sensitive information**, because other users cannot read or modify them.

Task 6 — chmod Practical

Apply chmod 777, 644, and 600 to project.txt and explain results.

```
sreelekha@LAPTOP-CKI0PC9R:~$ touch project.txt
sreelekha@LAPTOP-CKI0PC9R:~$ chmod 777 project.txt
sreelekha@LAPTOP-CKI0PC9R:~$ ls -l project.txt
-rwxrwxrwx 1 sreelekha sreelekha 0 Oct  1 11:35 project.txt
sreelekha@LAPTOP-CKI0PC9R:~$ chmod 644 project.txt
sreelekha@LAPTOP-CKI0PC9R:~$ ls -l project.txt
-rw-r--r-- 1 sreelekha sreelekha 0 Oct  1 11:35 project.txt
sreelekha@LAPTOP-CKI0PC9R:~$ chmod 600 project.txt
sreelekha@LAPTOP-CKI0PC9R:~$ ls -l project.txt
-rw----- 1 sreelekha sreelekha 0 Oct  1 11:35 project.txt
sreelekha@LAPTOP-CKI0PC9R:~$
```

1) Chmod 777-

Owner, group, and others all have **read, write, and execute** permissions.

2) chmod 644-

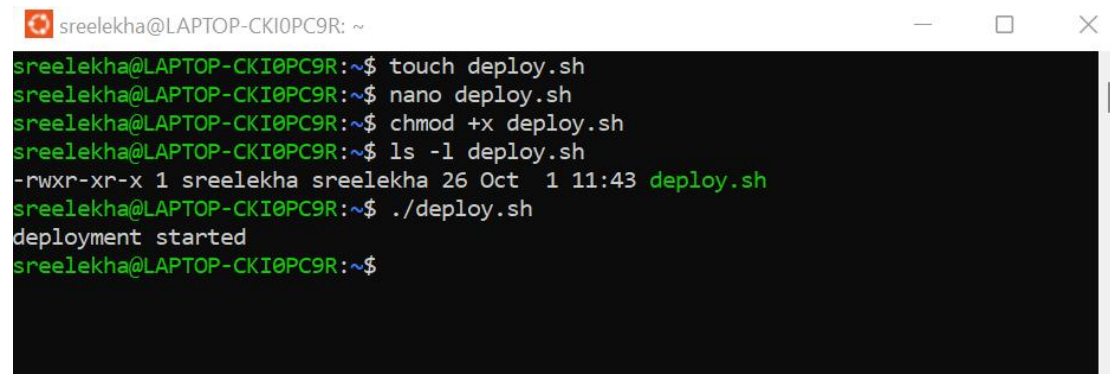
- Owner → Read + Write
- Group → Read
- Others → Read

3) chmod 600-

- Owner → Read + Write
- Group → No permission
- Others → No permission

Task 7 — Script Execution

Create `deploy.sh`, run `chmod +x deploy.sh`.



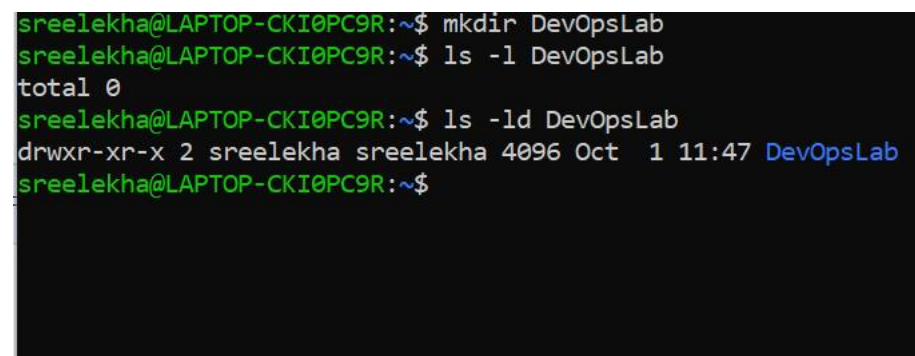
```
sreelekha@LAPTOP-CKI0PC9R: ~  
sreelekha@LAPTOP-CKI0PC9R:~$ touch deploy.sh  
sreelekha@LAPTOP-CKI0PC9R:~$ nano deploy.sh  
sreelekha@LAPTOP-CKI0PC9R:~$ chmod +x deploy.sh  
sreelekha@LAPTOP-CKI0PC9R:~$ ls -l deploy.sh  
-rwxr-xr-x 1 sreelekha sreelekha 26 Oct  1 11:43 deploy.sh  
sreelekha@LAPTOP-CKI0PC9R:~$ ./deploy.sh  
deployment started  
sreelekha@LAPTOP-CKI0PC9R:~$
```

Explain execute permission and DevOps usage.

Execute permission allows a script to run as a program. In DevOps, it is commonly used to run automation and deployment scripts.

Task 8 — Directory Permissions

Create `DevOpsLab` and analyze `drwxr-xr-x`.



```
sreelekha@LAPTOP-CKI0PC9R:~$ mkdir DevOpsLab  
sreelekha@LAPTOP-CKI0PC9R:~$ ls -l DevOpsLab  
total 0  
sreelekha@LAPTOP-CKI0PC9R:~$ ls -ld DevOpsLab  
drwxr-xr-x 2 sreelekha sreelekha 4096 Oct  1 11:47 DevOpsLab  
sreelekha@LAPTOP-CKI0PC9R:~$
```

Analyze `drwxr-xr-x`

- `d` → Directory
- `rw` → Owner: Read + Write + Execute
- `r-x` → Group: Read + Execute
- `r-x` → Others: Read + Execute

So:

Owner: `rw`

Group: `r-x`

Others: `r-x`

Explain ls -l vs ls -ld and directory execute permission.

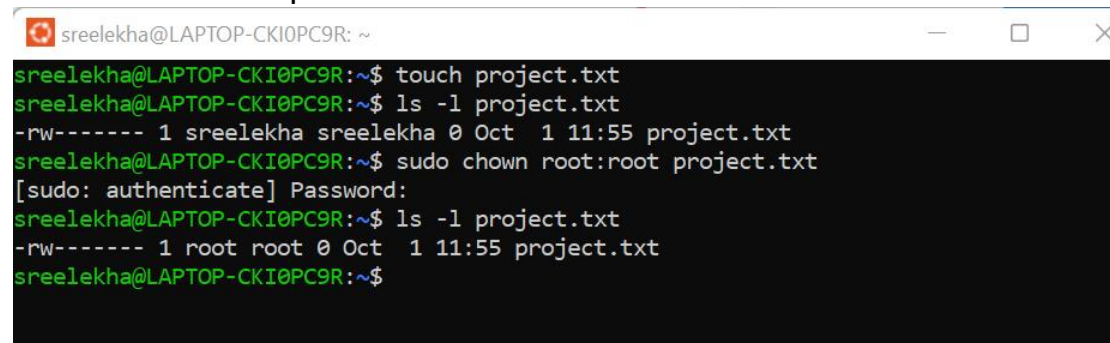
ls -l: Shows the contents inside the directory.

ls -ld: Shows the directory itself and its permissions.

For a directory, **execute permission means you can enter/access the directory and access items inside it when other required permissions are present.**

Task 9 — Ownership Management

Use chown and explain difference between chmod and chown.



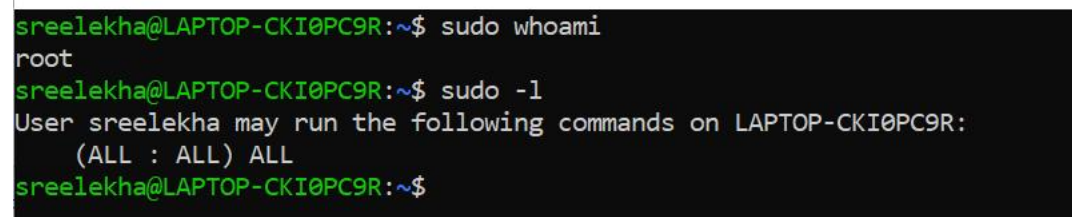
```
sreelekha@LAPTOP-CKI0PC9R: ~  
sreelekha@LAPTOP-CKI0PC9R:~$ touch project.txt  
sreelekha@LAPTOP-CKI0PC9R:~$ ls -l project.txt  
-rw----- 1 sreelekha sreelekha 0 Oct  1 11:55 project.txt  
sreelekha@LAPTOP-CKI0PC9R:~$ sudo chown root:root project.txt  
[sudo: authenticate] Password:  
sreelekha@LAPTOP-CKI0PC9R:~$ ls -l project.txt  
-rw----- 1 root root 0 Oct  1 11:55 project.txt  
sreelekha@LAPTOP-CKI0PC9R:~$
```

Chmod: change the permissions of files/directory

Chown: change the owner/group of the file/directory

Task 10 — Sudo Verification

Run sudo whoami and sudo -l.



```
sreelekha@LAPTOP-CKI0PC9R:~$ sudo whoami  
root  
sreelekha@LAPTOP-CKI0PC9R:~$ sudo -l  
User sreelekha may run the following commands on LAPTOP-CKI0PC9R:  
  (ALL : ALL) ALL  
sreelekha@LAPTOP-CKI0PC9R:~$
```

Sudo whoami → Shows root because the command runs with root privileges.

sudo -l → Shows the sudo permissions available to the current user.

sudo → Allows users to execute commands with administrative privileges without logging in directly as root.

Explain sudo and why it is preferred over root login.

Sudo= SuperUser Do

It allows a normal user to execute specific commands with **administrator/root privileges**.

Why is sudo preferred over root login?

1. **Better security** – You do not stay logged in as root.
2. **Limited privileges** – Administrative access is used only when needed.
3. **Accountability** – Sudo activity can be logged and audited.
4. **Less risk** – Accidental changes to the system are less likely when working as a normal user.

Real DevOps Scenario

deploy.sh → executable

config.env → owner only

application.log → owner write, others read

Assign correct permissions and justify.

```
sreelekha@LAPTOP-CKI0PC9R:~$ chmod 755 deploy.sh
sreelekha@LAPTOP-CKI0PC9R:~$ ls -l deploy.sh
-rwxr-xr-x 1 sreelekha sreelekha 26 Oct  1 11:43 deploy.sh
sreelekha@LAPTOP-CKI0PC9R:~$ chmod 600 config.env
chmod: cannot access 'config.env': No such file or directory
sreelekha@LAPTOP-CKI0PC9R:~$ touch config.env
sreelekha@LAPTOP-CKI0PC9R:~$ chmod 600 config.env
sreelekha@LAPTOP-CKI0PC9R:~$ ls -l config.env
-rw----- 1 sreelekha sreelekha 0 Oct  1 12:09 config.env
sreelekha@LAPTOP-CKI0PC9R:~$ touch application.log
sreelekha@LAPTOP-CKI0PC9R:~$ chmod 644 application.log
sreelekha@LAPTOP-CKI0PC9R:~$ ls -l application.log
-rw-r--r-- 1 sreelekha sreelekha 0 Oct  1 12:10 application.log
sreelekha@LAPTOP-CKI0PC9R:~$ ls -l deploy.sh config.env application.log
-rw-r--r-- 1 sreelekha sreelekha  0 Oct  1 12:10 application.log
-rw----- 1 sreelekha sreelekha  0 Oct  1 12:09 config.env
-rwxr-xr-x 1 sreelekha sreelekha 26 Oct  1 11:43 deploy.sh
sreelekha@LAPTOP-CKI0PC9R:~$
```

Justification: The owner can read, write, and execute the deployment script. Group and others can read and execute it.

Justification: Configuration files may contain sensitive information such as passwords, API keys, or database credentials. Only the owner should be able to read or modify it.

Justification: The owner can read and write the log file. Group and others can only read it, which allows log monitoring without allowing them to modify the log.

Bonus Challenge:--

Create DevOps-Team directory with:

deploy.sh → 755

app.log → 644

config.env → 600

Verify using ls -l

```
sreelekha@LAPTOP-CKI0PC9R:~$ mkdir DevOps-Team
sreelekha@LAPTOP-CKI0PC9R:~$ cd DevOps-Team
sreelekha@LAPTOP-CKI0PC9R:~/DevOps-Team$ touch deploy.sh app.log config.env
sreelekha@LAPTOP-CKI0PC9R:~/DevOps-Team$ chmod 755 deploy.sh
sreelekha@LAPTOP-CKI0PC9R:~/DevOps-Team$ chmod 644 app.log
sreelekha@LAPTOP-CKI0PC9R:~/DevOps-Team$ chmod 600 config.env
sreelekha@LAPTOP-CKI0PC9R:~/DevOps-Team$ ls -l
total 0
-rw-r--r-- 1 sreelekha sreelekha 0 Oct  1 12:16 app.log
-rw----- 1 sreelekha sreelekha 0 Oct  1 12:16 config.env
-rwxr-xr-x 1 sreelekha sreelekha 0 Oct  1 12:16 deploy.sh
sreelekha@LAPTOP-CKI0PC9R:~/DevOps-Team$
```